STOP. Read this entire prompt before making any changes. This is a single comprehensive update to OrderTicket.tsx and StrategyBuilder.tsx. Do not skip sections. Do not improvise. Do not add features not listed here. Do not remove existing functionality. Apply every section exactly as described.

These two files are the checkout/order ticket pages for a trading app. OrderTicket.tsx handles single stock orders, single option orders, and multi-leg option orders passed in from StrategyBuilder.tsx. StrategyBuilder.tsx is the multi-leg options builder with strategy templates, leg editing, and its own risk overview panel.

After all changes are applied, both files must compile with zero TypeScript errors and all existing functionality (order submission, risk checks, AI co-pilot, review modal, success/error states) must still work exactly as before.

═══

SECTION 1: TEXT COLOR HIERARCHY (BOTH FILES)

═══

The current problem: everything is the same flat white color. There is no visual hierarchy. Fix the color constants at the top of BOTH files to create three distinct tiers:

Tier 1 - PRIMARY (white): prices, values, quantities, the main data the user cares about
Tier 2 - SECONDARY (light warm gray): labels, section headers, row names
Tier 3 - MUTED (medium gray): hints, captions, placeholder text, helper text

In OrderTicket.tsx, find and replace these constants:

```
const TEXT = "#9ba1b5";
```
CHANGE TO:
```
const TEXT = "#b8bcc8";
```

```
const MUTED = "#6b7184";
```
CHANGE TO:
```
const MUTED = "#7d8494";
```

```
const DIM = "#6b7184";
```
CHANGE TO:
```
const DIM = "#7d8494";
```

```
const WHITE = "#f7f8fa";
```
KEEP AS IS - this is Tier 1 and is correct.

Do the exact same replacements in StrategyBuilder.tsx.

After this change, make sure:
- All price values, quantity numbers, and limit prices use color WHITE (#f7f8fa)
- All labels like "Order type", "Time in force", "Limit price", "Quantity", "Side", section headers like "RISK & INSIGHTS", "ORDER TICKET" use color TEXT (#b8bcc8)
- All helper text, hints, captions, and secondary info use color MUTED (#7d8494)

Scan through both files and fix any place where a label or section header incorrectly uses WHITE. Labels should be TEXT, not WHITE. Values should be WHITE, not TEXT.

Accent colors remain untouched:
- Green (#2ecc71 / UP) for buy actions, positive values, passing risk checks
- Red (#ff4b5c / DOWN) for sell actions, negative values, failing risk checks
- Gold (#f5a623 / GOLD) for warnings, highlighted pills, the CTA button, active states
- These accent colors are what create visual interest and draw attention to important elements

═══

SECTION 2: FONT SIZE - BUMP BY 2px, CAP AT 18px (BOTH FILES)

═══

In both OrderTicket.tsx and StrategyBuilder.tsx, do a systematic search and replace on ALL Tailwind text size classes:

text-[10px] -> text-[12px]
text-[11px] -> text-[13px]
text-[12px] -> text-[14px]
text-[13px] -> text-[15px]
text-[14px] -> text-[16px]
text-[15px] -> text-[17px]
text-[16px] -> text-[18px]

Do NOT change text-[17px] or text-[18px]. They stay as is.

Also bump inline style fontSize values by 2, but cap at 18:
fontSize: 9 -> fontSize: 11
fontSize: 10 -> fontSize: 12
fontSize: 11 -> fontSize: 13
fontSize: 12 -> fontSize: 14
fontSize: 13 -> fontSize: 15
fontSize: 14 -> fontSize: 16
fontSize: 16 -> fontSize: 18

Do NOT change fontSize: 17, 18, 20, or 22. Those are already large display sizes for prices.

Do NOT change any fontWeight values. The thin/light font is intentional. Do not add font-bold, font-semibold, or font-medium anywhere. Keep existing fontWeight: 300 on the body, and all other existing fontWeight values as they are.

═══

SECTION 3: REMOVE GRAY CARD BOXES ON FORM SECTIONS (OrderTicket.tsx)

═══

The stock order form currently wraps the Side/Quantity/Order Type/Limit Price section in a card with background CARD_SOFT, border, and borderRadius. This creates an unnecessary gray box. Find the single-leg form card container (it is the else branch after the isMultiLeg && strategyLegs check, roughly around line 797). It looks like:

<div style={{ background: CARD_SOFT, borderRadius: R_CARD, border: `1px solid ${BORDER}`, padding: "10px 12px" }}>

Change it to:

<div style={{ padding: "10px 12px 0" }}>

This removes the gray box and border but keeps the padding. The form fields inside will flow cleanly with the page background.

Similarly, for the instrument header card at the top (the one showing symbol, price, range, volume), keep it as a card - that one is fine as a distinct visual block.

For the multi-leg options ticket, the Legs card and the net credit price section can keep their card styling since they contain complex structured data.

═══

SECTION 4: ADD ALL SCHWAB ORDER TYPES (OrderTicket.tsx)

═══

Find the ORDER_TYPES array (around line 19) and the OrderType type (around line 13).

Replace the OrderType type with:
type OrderType = "MARKET" | "LIMIT" | "STOP" | "STOP_LIMIT" | "TRAILING_STOP" | "TRAILING_STOP_LIMIT" | "MARKET_ON_CLOSE" | "LIMIT_ON_CLOSE";

Replace the ORDER_TYPES array with:

```
const ORDER_TYPES: { value: OrderType; label: string; short: string }[] = [
  { value: "MARKET", label: "Market", short: "MKT" },
  { value: "LIMIT", label: "Limit", short: "LMT" },
  { value: "STOP", label: "Stop", short: "STP" },
  { value: "STOP_LIMIT", label: "Stop Limit", short: "STP LMT" },
  { value: "TRAILING_STOP", label: "Trail Stop", short: "TRAIL" },
  { value: "TRAILING_STOP_LIMIT", label: "Trail Stop Limit", short: "TRAIL LMT" },
  { value: "MARKET_ON_CLOSE", label: "MOC", short: "MOC" },
  { value: "LIMIT_ON_CLOSE", label: "LOC", short: "LOC" },
];
```

Update the needsLimit, needsStop, and needsTrail computed values to handle the new types:

```
const needsLimit = orderType === "LIMIT" || orderType === "STOP_LIMIT" ||
orderType === "TRAILING_STOP_LIMIT" || orderType === "LIMIT_ON_CLOSE";
const needsStop = orderType === "STOP" || orderType === "STOP_LIMIT" || orderType
=== "TRAILING_STOP_LIMIT";
const needsTrail = orderType === "TRAILING_STOP" || orderType ===
"TRAILING_STOP_LIMIT";
```

No changes needed to buildSchwabOrder since Schwab accepts these order type strings directly.

═══

SECTION 5: ADD ALL SCHWAB TIF OPTIONS (OrderTicket.tsx)

═══

Find the Duration type (around line 14) and the DURATIONS array (around line 27).

Replace the Duration type with:

```
type Duration = "DAY" | "GOOD_TILL_CANCEL" | "FILL_OR_KILL" | "SEAMLESS" |
"GOOD_TILL_CANCEL_EXT" | "AM" | "PM";
```

Replace the DURATIONS array with:

```
const DURATIONS: { value: Duration; label: string }[] = [
  { value: "DAY", label: "Day" },
  { value: "GOOD_TILL_CANCEL", label: "GTC" },
  { value: "FILL_OR_KILL", label: "FOK" },
  { value: "SEAMLESS", label: "EXT" },
  { value: "GOOD_TILL_CANCEL_EXT", label: "GTC+EXT" },
  { value: "AM", label: "AM" },
  { value: "PM", label: "PM" },
];
```

In the buildSchwabOrder function, find this line:

```
  session: extendedHours ? "SEAMLESS" : "NORMAL",
```

Replace with:
  session: (extendedHours || duration === "SEAMLESS" || duration === "GOOD_TILL_CANCEL_EXT") ? "SEAMLESS" : "NORMAL",

Also update the duration value that gets sent to Schwab. When duration is SEAMLESS or GOOD_TILL_CANCEL_EXT, Schwab expects the duration to be DAY or GOOD_TILL_CANCEL respectively with the session set to SEAMLESS. So add this mapping:

  duration: duration === "SEAMLESS" ? "DAY" : duration === "GOOD_TILL_CANCEL_EXT" ? "GOOD_TILL_CANCEL" : duration,

Apply the same session logic change in StrategyBuilder.tsx if it has its own buildSchwabOrder or order submission function.

═══
SECTION 6: ADVANCED SETTINGS - FULL SCHWAB FEATURE SET (OrderTicket.tsx)
═══

The Advanced Settings section currently shows a 3-column grid of small gray boxes for Effect, Exchange, and Ext Hrs. Replace the entire advancedOpen && conditional block with a clean list of rows using subtle dividers instead of boxes.

Add these state variables near the other useState declarations if they do not already exist:
  const [instruction, setInstruction] = useState("NONE");
  const [exchange, setExchange] = useState("BEST");
  const [taxLot, setTaxLot] = useState("Default");

Replace the advancedOpen && block with:

```
{advancedOpen && (
  <div style={{ padding: "0 12px" }}>
    <div className="flex items-center justify-between py-3" style={{ borderBottom: `1px solid ${DIVIDER}` }}>
      <span className="text-[13px]" style={{ color: TEXT }}>Position Effect</span>
      <button onClick={() => setPosEffect(posEffect === "OPENING" ? "CLOSING" : posEffect === "CLOSING" ? "AUTO" : "OPENING")} style={{ color: WHITE, background: "none", border: "none", padding: 0, fontSize: 14 }}>
        {posEffect === "AUTO" ? "Auto" : posEffect === "OPENING" ? "To Open" : "To Close"}
      </button>
    </div>
    <div className="flex items-center justify-between py-3" style={{ borderBottom: `1px solid ${DIVIDER}` }}>
      <span className="text-[13px]" style={{ color: TEXT }}>Instruction</span>
      <button onClick={() => setInstruction(instruction === "NONE" ? "ALL_OR_NONE"
```

```
: "NONE")} style={{ color: WHITE, background: "none", border: "none", padding: 0,
fontSize: 14 }}>
        {instruction === "NONE" ? "None" : "AON"}
      </button>
    </div>
    <div className="flex items-center justify-between py-3" style={{ borderBottom:
`1px solid ${DIVIDER}` }}>
      <span className="text-[13px]" style={{ color: TEXT }}>Exchange</span>
      <button onClick={() => setExchange(exchange === "BEST" ? "ARCA" : exchange
=== "ARCA" ? "NASDAQ" : "BEST")} style={{ color: WHITE, background: "none",
border: "none", padding: 0, fontSize: 14 }}>
        {exchange}
      </button>
    </div>
    <div className="flex items-center justify-between py-3" style={{ borderBottom:
`1px solid ${DIVIDER}` }}>
      <span className="text-[13px]" style={{ color: TEXT }}>Tax Lot Method</span>
      <button onClick={() => {
        const opts = ["Default", "FIFO", "LIFO", "High Cost", "Low Cost", "Best
Tax"];
        const idx = opts.indexOf(taxLot);
        setTaxLot(opts[(idx + 1) % opts.length]);
      }} style={{ color: WHITE, background: "none", border: "none", padding: 0,
fontSize: 14 }}>
        {taxLot}
      </button>
    </div>
    <div className="flex items-center justify-between py-3">
      <span className="text-[13px]" style={{ color: TEXT }}>Extended Hours</span>
      <button
        onClick={(e) => { e.stopPropagation(); setExtendedHours(!extendedHours);
}}
        className="relative rounded-full transition-colors duration-200"
        style={{ width: 36, height: 20, background: extendedHours ? GOLD : BORDER,
border: "none" }}
      >
        <div className="absolute rounded-full transition-transform duration-200"
style={{ width: 16, height: 16, top: 2, background: extendedHours ? BG : DIM,
transform: extendedHours ? "translateX(16px)" : "translateX(2px)" }} />
      </button>
    </div>
  </div>
)}
```

Also add the instruction to the buildSchwabOrder function. Find the order object and add:

```
  instruction: instruction !== "NONE" ? instruction : undefined,
```

═══

SECTION 7: COLLAPSIBLE RISK CARD WITH SMART COLLAPSED STATE (OrderTicket.tsx)

═══

This is critical. The Risk & Insights card must be collapsible. When COLLAPSED it must still communicate the risk level clearly so the user knows whether to worry. When EXPANDED it shows full details.

Add state if not already present:

```
  const [riskOpen, setRiskOpen] = useState(false);
```

Find the Risk & Insights card (starts with the CARD_GRAD container around line 985).

COLLAPSED STATE should show:
1. "RISK & INSIGHTS" header with the Pre-trade risk pill (PASS/WARN/FAIL) - this already exists
2. The summary row "Order vs account size" / "Spread X%" - this already exists
3. A one-line smart summary that tells the user WHY the risk level is what it is, in plain language
4. A "View full risk details ▾" toggle button

EXPANDED STATE shows everything that is currently shown plus the toggle becomes "Hide details ▴".

Replace the entire Risk & Insights card JSX with:

```
<div style={{ background: CARD_GRAD, borderRadius: R_CARD, border: `1px solid ${overallRisk === "GREEN" ? `${UP}20` : overallRisk === "YELLOW" ? `${GOLD}20` : `${DOWN}30`}`, padding: "10px 12px" }}>
  <div className="flex items-center justify-between">
    <span className="text-[13px] uppercase tracking-[0.06em]" style={{ color: TEXT }}>Risk & insights</span>
    {preTradeEnabled && riskChecks.length > 0 && (
      <span className="text-[13px] px-2 py-0.5" style={{
        borderRadius: 16,
        border: `1px solid ${overallRisk === "GREEN" ? `${UP}66` : overallRisk === "YELLOW" ? `${GOLD}66` : `${DOWN}80`}`,
        color: levelColor(overallRisk),
        background: overallRisk === "GREEN" ? `${UP}0a` : overallRisk === "YELLOW" ? `${GOLD}0a` : `${DOWN}0a`,
      }}>
        Pre-trade risk · {overallRisk === "GREEN" ? "PASS" : overallRisk === "YELLOW" ? "WARN" : "FAIL"}
      </span>
    )}
```

```
  </div>

  {(() => {
    const spreadPct = quote?.bid != null && quote?.ask != null && quote.bid > 0
      ? (((quote.ask - quote.bid) / quote.bid) * 100).toFixed(1) + "%"
      : "—";
    return (
      <div className="mt-1.5 flex flex-wrap items-center justify-between gap-2 pt-
1.5 text-[13px]" style={{ borderTop: `1px dashed ${DIVIDER}`, color: TEXT }}>
        <span>Order vs account size</span>
        <span>Spread {spreadPct}</span>
      </div>
    );
  })()}

  {/* Smart collapsed summary - shows WHY the risk level is what it is */}
  {!riskOpen && preTradeEnabled && riskChecks.length > 0 && (
    <div className="mt-1.5 text-[13px]" style={{ color: levelColor(overallRisk) +
"cc" }}>
      {(() => {
        const reds = riskChecks.filter(c => c.level === "RED");
        const yellows = riskChecks.filter(c => c.level === "YELLOW");
        if (reds.length > 0) return `${reds.length} issue${reds.length > 1 ? "s" :
""}: ${reds.map(c => c.label).join(", ")}`;
        if (yellows.length > 0) return `${yellows.length} warning${yellows.length
> 1 ? "s" : ""}: ${yellows.map(c => c.label).join(", ")}`;
        return `All ${riskChecks.length} checks passed`;
      })()}
    </div>
  )}

  <button
    onClick={() => setRiskOpen(!riskOpen)}
    className="w-full flex items-center justify-between mt-2 pt-1.5 text-[13px]"
    style={{ borderTop: `1px dashed ${DIVIDER}`, color: TEXT, background: "none",
border: "none", borderTop: `1px dashed ${DIVIDER}`, padding: "6px 0 0" }}
  >
    <span>{riskOpen ? "Hide details" : "View full risk details"}</span>
    <span style={{ color: MUTED }}>{riskOpen ? "▲" : "▼"}</span>
  </button>

  {riskOpen && (
    <>
      {preTradeEnabled && riskChecks.length > 0 && (
        <div className="mt-2 space-y-2 text-[13px]">
          {riskChecks.filter(c => {
            // Hide irrelevant "Equity — n/a" checks on stock orders
```

```
            if (!isOption && c.detail.includes("Equity")) return false;
            return true;
          }).map(c => (
            <div key={c.id} className="flex gap-2">
              <span className="mt-1.5 h-2.5 w-2.5 rounded-full shrink-0" style={{
background: levelColor(c.level) }} />
              <div>
                <div style={{ color: WHITE, fontWeight: 400 }}>{c.label}</div>
                <div style={{ color: MUTED }}>{c.detail}</div>
              </div>
            </div>
          ))}
        </div>
      )}

      <AiCoPilotPanel
        side={side}
        symbol={symbol}
        limitPrice={parseFloat(limitPrice) || 0}
        bid={quote?.bid ?? null}
        ask={quote?.ask ?? null}
        quantity={quantity}
        isOption={isOption}
      />
    </>
  )}

  {/* When collapsed, hide the AiCoPilotPanel and individual risk checks */}
</div>
```

IMPORTANT: The AiCoPilotPanel that currently renders OUTSIDE the riskOpen conditional must be MOVED INSIDE it. When collapsed, only the header, summary row, smart summary line, and toggle button should be visible. Nothing else.

═══

SECTION 8: FILTER IRRELEVANT STOCK RISK CHECKS (OrderTicket.tsx)

═══

This is already handled in Section 7 above with the filter:
`riskChecks.filter(c => { if (!isOption && c.detail.includes("Equity")) return false; return true; })`

But also apply the same filter to the collapsed summary in Section 7. The smart summary should not count "Equity — n/a" items as warnings for stock orders.

Additionally, in the runPreTradeChecks function, the checks for Prob of Profit, Vol Environment, and DTE/Gamma Risk all push a check with detail "Equity — n/a"

when isOption is false. Instead of pushing these at all for stock orders, simply skip them. Find these three blocks (around lines 139-193) and wrap each one:

```
For Prob of Profit (around line 139):
  if (isOption) {
    // existing option logic
  }
  // Remove the else block that pushes "Equity — n/a"

For Vol Environment (around line 168):
  if (isOption) {
    // existing option logic
  }
  // Remove the else block that pushes "Equity — n/a"

For DTE / Gamma Risk (around line 184):
  if (isOption) {
    // existing option logic
  }
  // Remove the else block that pushes "Equity — n/a"
```

This means stock orders will only show: Pulse Alignment, Risk/Reward, Liquidity, and Position Size. No more useless "n/a" entries.

═══

SECTION 9: BALANCES CARD WITH REAL DATA (OrderTicket.tsx)

═══

The Balances card currently shows $25,000 because it reads from accountSize which is a settings default, not real account data.

Check if there is already a hook or API call that fetches real Schwab account balances. The app already has a Portfolio page that shows Net Liq Value, Available $, and Buying Power. Look for a portfolio hook, a balances hook, or an API endpoint like /api/portfolio/balances or /api/portfolio/account.

If such a hook exists, import it and use it. If not, add a useEffect that fetches account balances when the ticket opens:

```
  const [balances, setBalances] = useState<{ cashBalance: number; buyingPower:
number; optionBuyingPower: number; netLiqValue: number } | null>(null);

  useEffect(() => {
    if (!isOpen || !accountHash) return;
    fetchWithAuth(`/api/portfolio/balances?accountHash=${accountHash}`)
      .then(r => r.json())
      .then(d => {
```

```
        if (d && !d.error) setBalances(d);
      })
      .catch(() => {});
  }, [isOpen, accountHash]);
```

Then render the Balances card (placed between the Order Description card and the bottom of the scroll area):

```
<div style={{ background: CARD_GRAD, borderRadius: R_CARD, border: `1px solid
${BORDER}`, padding: "10px 12px" }}>
  <span className="text-[13px]" style={{ color: TEXT }}>Balances</span>
  <div className="mt-1.5 space-y-0">
    {[
      { label: "Cash & Sweep", value: balances ? fmtCurrency(balances.cashBalance)
: "—" },
      { label: isOption ? "Option Buying Power" : "Stock Buying Power", value:
balances ? fmtCurrency(isOption ? balances.optionBuyingPower :
balances.buyingPower) : "—" },
      { label: "Net Liq Value", value: balances ?
fmtCurrency(balances.netLiqValue) : "—" },
      { label: "BP After Trade", value: balances && estimatedCost != null ?
fmtCurrency(Math.max(0, (isOption ? balances.optionBuyingPower :
balances.buyingPower) - Math.abs(estimatedCost))) : "—" },
    ].map((row, i, arr) => (
      <div key={i} className="flex items-center justify-between py-2" style={{
borderBottom: i < arr.length - 1 ? `1px solid ${DIVIDER}` : "none" }}>
        <span className="text-[13px]" style={{ color: TEXT }}>{row.label}</span>
        <span className="text-[14px]" style={{ color: WHITE }}>{row.value}</span>
      </div>
    ))}
  </div>
</div>
```

If the /api/portfolio/balances endpoint does not exist, check what endpoint the Portfolio page uses and use that same one. The data is already being fetched elsewhere in the app. Find it and reuse it.

If you absolutely cannot find the right endpoint, fall back to accountSize from the store but label it "Buying Power (est.)" so the user knows it is approximate.

SECTION 10: ORDER DESCRIPTION CARD (OrderTicket.tsx)

Add an Order Description card between the Risk & Insights card and the Balances card. This shows a plain-English summary of exactly what will be submitted.

```
<div style={{ background: CARD_GRAD, borderRadius: R_CARD, border: `1px solid
${BORDER}`, padding: "10px 12px" }}>
  <span className="text-[13px]" style={{ color: TEXT }}>Order Description</span>
  <div className="mt-1 text-[15px] leading-relaxed" style={{ color: WHITE }}>
    {(() => {
      if (isMultiLeg && strategyLegs) {
        const legDescriptions = strategyLegs.map(leg => {
          const isBuyLeg = leg.instruction.startsWith("BUY");
          const dir = isBuyLeg ? "BUY" : "SELL";
          const qty = leg.quantity * quantity;
          return `${dir} ${qty > 0 ? "+" : ""}${isBuyLeg ? "+" : "-
"}${Math.abs(qty)} ${leg.strike} ${leg.optionType === "CALL" ? "Call" : "Put"}`;
        }).join(", ");
        const dur = DURATIONS.find(d => d.value === duration)?.label ?? duration;
        return `${strategyIsCredit ? "SELL" : "BUY"} ${symbol}
${strategyLegs.length}-leg strategy: ${legDescriptions}. ${needsLimit &&
limitPrice ? `Limit ${limitPrice}` : "Market"}, ${dur}${extendedHours ? " + Ext" :
""}.`;
      }
      const action = side;
      const asset = isOption ? "contract" : "share";
      const plural = quantity > 1 ? "s" : "";
      const priceStr = orderType === "MARKET" ? "at Market"
        : needsLimit && limitPrice ? `at ${limitPrice} Limit`
        : needsStop && stopPrice ? `at ${stopPrice} Stop`
        : needsTrail && trailOffset ? `Trail $${trailOffset}`
        : "";
      const dur = DURATIONS.find(d => d.value === duration)?.label ?? duration;
      return `${action} ${quantity} ${asset}${plural} ${displaySymbol}
${priceStr}, ${dur}${extendedHours ? " + Ext" : ""}`;
    })()}
  </div>
</div>
```

═══

SECTION 11: PAYOFF CHART - FILL THE DEAD BOX (StrategyBuilder.tsx)

═══

In StrategyBuilder.tsx, find the payoff chart placeholder. It is a bordered dashed box that shows "P/L at expiration vs underlying price" but is otherwise empty/black. It looks something like:

```
<div style={{ ... height: something, overflow: "hidden", border: "... dashed ...",
background: "..." }}>
```

Inside this container, after any existing content, add a gradient polygon that simulates a payoff curve:

```
<div style={{
  position: "absolute",
  inset: "12px",
  borderRadius: 8,
  background: `linear-gradient(to top right, ${UP}25, ${GOLD}20)`,
  clipPath: "polygon(0% 65%, 30% 65%, 70% 20%, 100% 20%, 100% 100%, 0% 100%)",
}} />
```

Make sure the parent container has position: "relative" if it does not already.

This creates a simple gradient shape that reads as a payoff curve without needing a real charting library. It is purely visual decoration.

==========

SECTION 12: FIX BOTTOM BAR OVERLAP (OrderTicket.tsx)

==========

The scrollable content area has pb-28 which is not enough padding. Content overlaps behind the sticky bottom bar.

Find: className="flex-1 overflow-y-auto pb-28"
Change to: className="flex-1 overflow-y-auto pb-40"

Find the sticky bottom bar background (the gradient that fades from transparent to the background color):
Find: background: `linear-gradient(to top, rgba(5,6,7,0.97), rgba(5,6,7,0.8), transparent)`
Change to: background: `linear-gradient(to top, ${BG}, ${BG}f5, transparent)`

This makes the bar fully opaque so text never bleeds through.

Apply the same two changes in StrategyBuilder.tsx if it has a similar sticky bottom bar.

==========

SECTION 13: RISK CHECK DOT COLORS (OrderTicket.tsx)

==========

Verify that the risk check severity dots use the levelColor() function which maps:
- GREEN -> UP (#2ecc71) - green dot
- YELLOW -> GOLD (#f5a623) - orange/gold dot
- RED -> DOWN (#ff4b5c) - red dot

Each risk check item should render a colored dot using:
  style={{ background: levelColor(c.level) }}

If the agent previously changed these dots to all be the same color, fix them. Each dot must reflect its individual check level, not the overall risk level. A PASS card might have 5 green dots and 1 yellow dot. A WARN card might have 3 green dots and 2 yellow dots. A FAIL card has at least 1 red dot. The dots must be INDIVIDUALLY colored.

The risk check label (like "Pulse Alignment", "Liquidity") should be in WHITE. The risk check detail (like "No Market Pulse data available", "Spread 1.9% — tight") should be in MUTED.

═══

SECTION 14: APPLY CHANGES TO StrategyBuilder.tsx

═══

StrategyBuilder.tsx needs all visual changes from Sections 1, 2, 11, 12, and 13.

For Section 1 (colors): same constant replacements.
For Section 2 (font sizes): same class replacements.
For Section 11 (payoff chart): already described above.
For Section 12 (bottom bar): same padding and opacity fix.
For Section 13 (risk dots): same verification.

StrategyBuilder.tsx has its own ORDER section at the bottom with Order Type and Time in Force dropdowns. If these dropdowns exist, expand them with the same options from Sections 4 and 5. If StrategyBuilder delegates order type and TIF to OrderTicket (by passing through strategyLegs), then leave it alone.

StrategyBuilder.tsx also has its own risk overview panel. Make it collapsible with the same pattern from Section 7.

═══

VERIFICATION CHECKLIST - DO NOT SKIP

═══

After ALL changes, verify each of these:

1. TypeScript compiles with zero errors in both files
2. OrderTicket opens correctly for stock orders (test with any stock symbol)
3. OrderTicket opens correctly for single option orders
4. OrderTicket opens correctly for multi-leg strategies from StrategyBuilder
5. StrategyBuilder opens and all 7 strategy templates work
6. All dropdown menus (Order Type, TIF) open, close, and persist selection
7. Order submission still works end-to-end (buildSchwabOrder produces valid payloads)
8. Review confirmation modal still shows all order details correctly
9. Risk card is COLLAPSED by default and shows a smart summary of issues
10. Risk card EXPANDS on tap to show individual checks with correctly colored dots

11. Stock orders do NOT show "Equity — n/a" risk checks
12. Font is larger but still thin/light weight throughout
13. Text has clear visual hierarchy: white values, light gray labels, medium gray hints
14. No text overlaps the sticky bottom bar on any screen
15. Advanced settings shows clean rows with dividers, not gray boxes
16. Payoff chart in StrategyBuilder has a gradient fill, not an empty black box
17. Accent colors (green for buy/pass, red for sell/fail, gold for warnings) are preserved and visible